BranchNorm: Robustly Scaling Extremely Deep Transformers

Yijin Liu[*], **Xianfeng Zeng**[*], **Fandong Meng**[†] **and Jie Zhou**
Pattern Recognition Center, WeChat AI, Tencent Inc, China
{yijinliu, xianfzeng, fandongmeng, withtomzhou}@tencent.com

arXiv:2305.02790v1 [cs.LG] 4 May 2023

Abstract

Recently, DeepNorm scales Transformers into extremely deep (*i.e.,* 1000 layers) and reveals the promising potential of deep scaling. To stabilize the training of deep models, DeepNorm (Wang et al., 2022a) attempts to constrain the model update to a constant value. Although applying such a constraint can benefit the early stage of model training, it may lead to undertrained models during the whole training procedure. In this paper, we propose BranchNorm, which dynamically rescales the non-residual branch of Transformer in accordance with the training period. BranchNorm not only theoretically stabilizes the training with smooth gradient norms at the early stage, but also encourages better convergence in the subsequent training stage. Experiment results on multiple translation tasks demonstrate that BranchNorm achieves a better trade-off between training stability and converge performance.

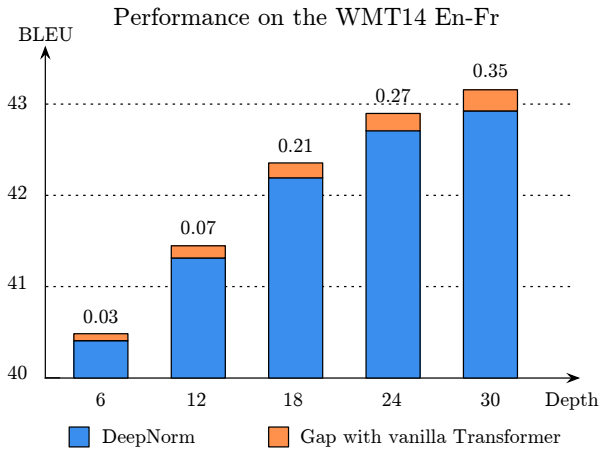

Figure 1: BLEU(%) scores on the WMT2014 En-Fr dataset after models fully converge. 'Gap' refers to the performance decline observed after applying DeepNorm on the vanilla Transformer.

1 Introduction

In recent years, Transformers (Vaswani et al., 2017) have been developed rapidly and achieved state-of-the-art (SOTA) performance on a wide range of tasks. Meanwhile, the model capacity gets substantially expanded by widening the model dimension (Devlin et al., 2019; Liu et al., 2019; Goyal et al., 2021; Lin et al., 2021; Smith et al., 2022). Given that deep neural models learn feature representations with multiple layers of abstraction (LeCun et al., 2015), it is more attractive to increase model capacity by scaling depths than widths. Unfortunately, due to the training instability of Transformers, the depths of these SOTA models are still relatively shallow (Kaplan et al., 2020; Hoffmann et al., 2022).

To stabilize the training of Transformers, there have been various efforts on better architectures (Wang et al., 2019; Shleifer et al., 2021; Wang et al., 2022b), or the implementation of proper initialization (Zhang et al., 2019a; Huang et al., 2020; Wang et al., 2022a). Among them, the most representative approach is DeepNorm (Wang et al., 2022a), which first scales Transformers to 1000 layers and significantly outperforms existing shallow counterparts.

Specifically, DeepNorm aims to constrain the model update to a constant level by upweighting the residual connections in Transformer and reducing the variance of parameter initialization. As a result, the stability of Transformers is improved in the early training stage. However, in the subsequent training stage, the limitation of the magnitude of parameter updates imposed by DeepNorm may ultimately yield undertrained models. To verify the above conjecture, we first conduct experiments on shallow Transformers to guarantee convergences. As shown in Figure 1, it is observed that DeepNorm brings a certain degree of performance decline on vanilla Transformers, and this issue tends to get

[*] Equal contributions.
[†] Corresponding author.

worse when models get deeper.

To address the above issue, we propose a simple yet effective approach to robustly scale extremely deep Transformers, named BranchNorm. Specifically, the non-residual branch[1] of the Transformer is dynamically rescaled in accordance with the training period. In the early stage of model training, BranchNorm theoretically stabilizes the training with smooth gradient norms. While in the subsequent training stage, BranchNorm progressively degenerates into vanilla Post-LayerNorm (*i.e.,* Post-LN) to promote better convergence. Experiments on a wide range of translation tasks show that BranchNorm brings consistent improvement over DeepNorm, and effectively alleviates the above undertrained issue. Moreover, BranchNorm performs more robustly on some key hyperparameters (*e.g.,* warmup) than DeepNorm, which makes it likely to be a portable alternative for scaling extremely deep Transformers.

The contributions of this paper can be summarized as follows:

- We propose a simple yet effective normalization approach, named BranchNorm, to stabilize the training of extremely deep Transformers.
- BranchNorm achieves a better trade-off between training stability and converges performance on a wide range of translation tasks.
- BranchNorm is demonstrated to alleviate the problem of parameter redundancy in extremely deep models, from the perspective of representing similarity and sparsity of activation functions.

2 Background

In this section, we first provide a brief overview of the difference between Post-LN and Pre-LN, and subsequently introduce the approach of DeepNorm.

Post-LN and Pre-LN. Firstly, Wang et al. (2019); Nguyen and Salazar (2019) observe that the position of LayerNorm (Ba et al., 2016) has a significant effect on training stability, and propose the more stable Pre-LN variant when compared with the original Post-LN (Vaswani et al., 2017). An example of these two architectures is shown in Figure 2. Subsequently, Liu et al. (2020b) further analyze that Pre-LN may have an excessive reliance on its residual connections, which inhibits the model from unleashing its full potential. Motivated by the above observation, we base our approach on Post-LN in the remainder of our experiments.

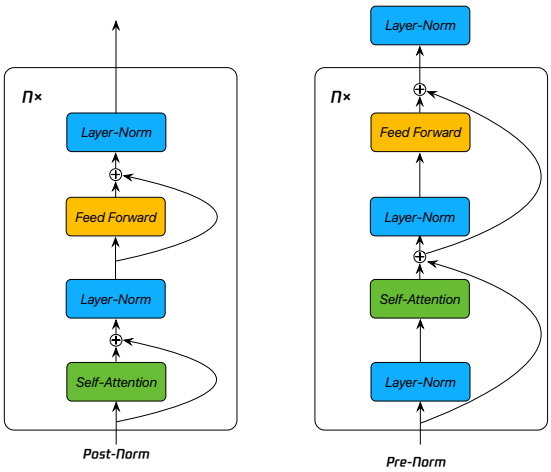

Figure 2: The architectures of Pre-Norm (*i.e.,* Pre-LN) and Post-Norm (*i.e.,* Post-LN) Transformers.

Formally, given the input of the sub-layer in the l-th sub-layer x_l, calculates the output x_{l+1} is calculated by Post-LN as follows:

$$x_{l+1} = LN(x_l + \mathcal{F}(x_l; \theta_l)) \tag{1}$$

where LN is an abbreviation for LayerNorm[2], $\mathcal{F}$ represents the function of the current sub-layer (attention or feed-forward) and θ_l denotes the corresponding parameters of the sub-layer.

DeepNorm. DeepNorm follows the Post-LN Transformer architecture and rescales the residual branch with a scalar factor $\alpha > 1$. Similarly, the l-th sub-layer is calculated by DeepNorm as follows:

$$x_{l+1} = LN(\alpha x_l + \mathcal{F}(x_l; \theta_l)) \tag{2}$$

In addition, DeepNorm reduces the variance of the initial parameters by scaling factor $\beta < 1$. Both the α and β are functions of model depths, which are derived from the assumption of constant model update. For a strandard Transformer with N-layer encoder and M-layer decoder, DeepNorm calculate

[1]Note that we name the residual connections in Transformer as 'residual branch' and the other branch as 'non-residual branch' in this paper.

[2]For brevity, the two learnable parameters in LayerNorm are omitted.

α and β as follows:

$$\begin{aligned}
\alpha_{encoder} &= 0.81(N^4M)^{\frac{1}{16}} \\
\beta_{encoder} &= 0.87(N^4M)^{-\frac{1}{16}} \\
\alpha_{decoder} &= (3M)^{\frac{1}{4}} \\
\beta_{decoder} &= (12M)^{-\frac{1}{4}}
\end{aligned} \tag{3}$$

Note that β merely affects the model initialization, while α is used and fixed during the whole procedure. Moreover, with the model getting deeper, DeepNorm assigns larger value of α, which leads to the model outputs x_{l+1} depend too much on the residual branch αx_l and thus ultimately yields the undertrained model parameters θ_l in Equation (2).

3 Approaches

In this section, we first analyze the instability of Post-LN from the perspective of gradient norm, then demonstrate how DeepNorm can alleviate the unbalanced gradients to a certain extent, and finally introduce our proposed method BranchNorm.

3.1 Perspective of Gradient

Unbalanced gradients are mainly responsible for the instability of Transformer[3] (Wang et al., 2019; Shleifer et al., 2021; Zhu et al., 2021), we firstly explore the relation between gradient and model depth following Wang et al. (2019). Given a Transformer with L sub-layers and the training loss $\mathcal{E}$, the gradient for the l-th sub-layer is calculated by the chain rule[4]:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(\mathcal{F}\left(x_k;\theta_k\right)\right)}{\partial \mathcal{F}\left(x_k;\theta_k\right)}}_{LN} \times \underbrace{\prod_{k=l}^{L-1}\left(1+\frac{\partial \mathcal{F}\left(x_k;\theta_k\right)}{\partial x_k}\right)}_{residual} \tag{4}$$

where the gradient consists of three terms and the last two items are multiplicative with respect to the number of model layers L. Once L gets larger, the values of the last two items may become very large or very small, which can yield the gradient vanishing or exploding.

Similarly, we analyze the gradient of DeepNorm based on Equation (2), and get the gradient of the l-th sub-layer is calculated by:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1}\left(\frac{\partial \mathrm{LN}\left(\alpha x_k+\mathcal{F}\left(x_k;\theta_l\right)\right)}{\partial\left(\alpha x_k+\mathcal{F}\left(x_k;\theta_l\right)\right)}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1}\left(\alpha+\frac{\partial \mathcal{F}\left(x_k;\theta_k\right)}{\partial x_k}\right)}_{residual} \tag{5}$$

In DeepNorm, α increases with model depth and helps with training stability. Theoretically, α can go to infinity to represent the upper bound of DeepNorm's stability. Here, we introduce this assumption to simplify the derivation: If α get large enough, *i.e.,* $\alpha \to \infty$, the LN item can be approximated as $\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}(\alpha x_k)}{\partial(\alpha x_k)}$, and the residual item can be approximated as $\prod_{k=l}^{L-1} \alpha$, we put them into Equation (5) and can simplify it as follows:

$$\begin{aligned}
\frac{\partial \mathcal{E}}{\partial x_l} &\approx \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1}\left(\frac{\partial \mathrm{LN}\left(\alpha x_k\right)}{\partial\left(\alpha x_k\right)}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \alpha}_{residual} \\
&= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1}\left(\frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k} \cdot \frac{1}{\alpha}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \alpha}_{residual} \\
&= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k}}_{LN} \quad (\alpha \to \infty)
\end{aligned} \tag{6}$$

When compared to the gradient of Post-LN in Equation (4), DeepNorm can approximately eliminate the final residual item, and thus effectively mitigate the risk of gradient vanishing or exploding. Although a larger α in DeepNorm results in more stable gradients, it may come at the expense of final convergence performance, as mentioned above. Considering that the unbalanced gradients generally occur during the early training stage, it may be more appropriate if α can be varied based on the training period.

3.2 BranchNorm

In this section, we summarize the observations from previous sections and introduce BranchNorm:

$$x_{l+1} = LN(x_l + \alpha \mathcal{F}\left(x_l;\theta_l\right)) \tag{7}$$

[3] In recent years, there are also researchers questioning this point and providing different perspectives (Liu et al., 2020b; Wang et al., 2022a). Given that more explorations and discussions are needed to make it out, we still conduct analysis from the perspective of gradient norm in this paper.

[4] More detailed derivations are in Appendix A.

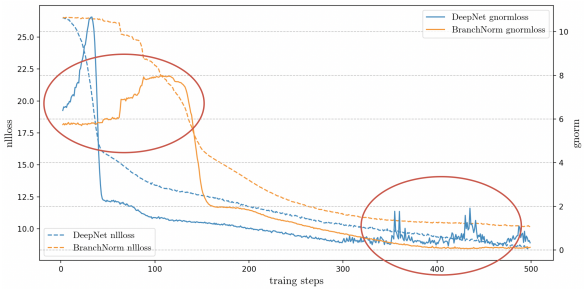

Figure 3: Gradient norm (solid line) and negative log likelihood loss (nllloss, dotted line) at the very beginning of training.

It is analogous to the dual form of DeepNorm, but with two key differences: First, BranchNorm utilizes a dynamic factor (*i.e.,* α) that allows it to normalize gradients during the early training stage and gradually eliminate the negative effects of these normalizations during the later stage. Second, the α of BranchNorm scales on the non-residual branches in Transformer, allowing for exactly normalize early stage's gradients without the strong assumptions of DeepNorm in Equation (5). Specifically, α in Equation (7) is a simple factor with respect to the number of training step t. Here, we use a simple linear incremental approach:

$$\alpha_t = \min(1.0, t/T) \tag{8}$$

where T is the maximum number of steps to conduct BranchNorm. At the very beginning of training, α_t is approaching 0, which means that the model approximates the constant transformation and gets updated with smooth gradients. Following the analysis in the above section, we get the gradient of BranchNorm for the l-th sub-layer:

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial x_l} &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k + \alpha \mathcal{F}\left(x_k; \theta_l\right)\right)}{\partial \left(x_k + \alpha \mathcal{F}\left(x_k; \theta_l\right)\right)}}_{LN} \times \\ &\quad \underbrace{\prod_{k=l}^{L-1} \left(1 + \alpha \frac{\partial \mathcal{F}\left(x_k; \theta_l\right)}{\partial x_k}\right)}_{residual} \\ &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k}}_{LN} \quad (\alpha = 0)\end{aligned} \tag{9}$$

At the very beginning of training, BranchNorm can stabilize the gradient norm while DeepNorm requires a relatively strong assumption ($\alpha \to \infty$) in Equation (6). Experimentally, as shown in Figure 3, we observe corresponding smoother gradients of BranchNorm at the very beginning of training. Once the training step t reaches the predefined maximum step T, BranchNorm degenerates to the vanilla Post-LN to achieve better convergence. We further validate the hyperparameter insensitivity of BranchNorm in Section 5.1.

4 Experiments and Results

We conduct extensive experiments on both bilingual translation and multilingual translation tasks to verify our approach. In this section, we will describe our experimental settings and present results.

4.1 Datasets and Evaluations

We use the standard WMT 2017 English-German (En-De), WMT 2014 English-French (En-Fr), and IWSLT 2014 German-English (De-En) datasets for the bilingual task, which is processed following the official scripts of fairseq[5]. For the multilingual task, we conduct experiments on the OPUS-100 (Zhang et al., 2020) and MultiUN (Gu et al., 2019) dataset and follow the corresponding processing in existing studies.

For evaluation, we set the beam size to 4 and the length penalty to 0.6 during inference. We use the *multibleu.perl* to calculate cased sensitive BLEU scores for WMT 2017 En-De[6] and WMT 2014 En-Fr. Besides, we use sacreBLEU[7] to calculate cased sensitive BLEU scores for OPUS-100 and cased insensitive BLEU scores for MultiUN following Wang et al. (2021).

4.2 Training Settings

Our experiments are based on the fairseq codebase (Ott et al., 2019). For all experiments, we use the standard Transformer base setting which sets hidden dimensions to 512 and feed-forward inner representation to 2048, if not specifically noted. We initialize model parameters following DeepNorm (Wang et al., 2022a). All experiments are conducted on 32 NVIDIA A100 GPUs where each is allocated with a batch size of approximately 16,384 tokens. All Transformer models are trained for 100k steps with the early stop for small-scale

[5]https://github.com/facebookresearch/fairseq

[6]For a rigorous comparison, we use the same test set with DeepNorm, *i.e., newstest2014.*

[7]https://github.com/mjpost/sacrebleu

Models	LN	6L-6L	18L-18L	50L-50L	100L-100L	250L-250L
Vanilla Post-LN (Vaswani et al., 2017)	Post	28.1		diverged		
DS-Init (Zhang et al., 2019a)	Post	27.9		diverged		
Admin (Liu et al., 2020b)	Post	27.9	28.8		diverged	
ReZero (Bachlechner et al., 2020)	No	26.9		diverged		
R-Fixup (Zhang et al., 2019b)	No	27.5	28.4	27.7	diverged	diverged
T-Fixup (Huang et al., 2020)	No	27.5	28.4	27.9	diverged	diverged
Vanilla Pre-LN (Vaswani et al., 2017)	Pre	27.0	28.1	28.0	27.4	27.5
DLCL (Wang et al., 2019)	Pre	27.4	28.2	diverged	27.5	27.7
NormFormer (Shleifer et al., 2021)	Pre	27.0	28.3	27.8	diverged	diverged
Sub-LN (Wang et al., 2022b) †	Pre	27.5	28.3	28.7	27.7	27.9
DeepNorm (Wang et al., 2022a)	Post	27.8	28.8	29.0	28.9	–
DeepNorm (Wang et al., 2022a) †	Post	28.6	29.1	29.7	29.3	29.0
BranchNorm (ours)	Post	**29.3**	**30.3**	**30.7***	**29.8**	**29.6**

Table 1: BLEU scores (%) on the WMT-17 En-De test set with depth-scaling. † indicates our reimplementations. AL-BL refers to a Transformer with A-layer encoder and B-layer decoder. '*' means BranchNorm is significantly better than DeepNorm with $p < 0.03$.

Models	LN	6L-6L	18L-18L	50L-50L	100L-100L	250L-250L	500L-500L
Vanilla Post-LN (2017)	Pre	41.48	43.27		diverged		
Vanilla Pre-LN (2017)	Pre	40.96	42.48	42.70	43.12	43.25	43.18
DLCL (2019)	Pre	41.33	42.81	43.05		diverged	
Sub-LN (2022b) †	Pre	41.12	42.68	43.28	43.31	43.42	43.21
DeepNorm (2022a) †	Post	41.47	42.92	43.79	43.93	43.87	43.67
MixNorm (ours)	Post	**41.96**	43.34	43.81	43.91	43.73	43.41
BranchNorm (ours)	Post	41.67	**43.53**	**43.89**	**44.20**	**44.30***	**44.27**

Table 2: BLEU scores (%) on the WMT-14 En-Fr test set with depth-scaling. † indicates our reimplementations. AL-BL refers to a Transformer with A-layer encoder and B-layer decoder. '*' means BranchNorm is significantly better than DeepNorm with $p < 0.03$.

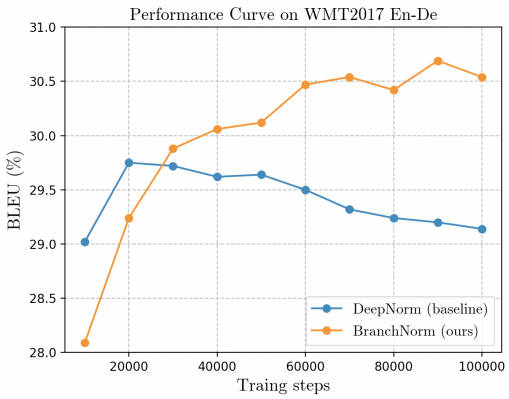

Figure 4: BLEU score curve of 50L-50L models on the WMT 2017 En-De with the increase of training steps.

datasets. The maximum norm step T of BranchNorm in Equation (8) is set to 4,000 for all experiments. More details are elaborated in Appendix B.

4.3 Bilingual Translation Tasks

We compare several state-of-the-art approaches for deep Transformers, including DeepNorm (Wang et al., 2022a), Sub-LN (Wang et al., 2022b), NormFormer (Shleifer et al., 2021), ReZero (Bachlechner et al., 2020) and *etc.* We implemented DeepNorm and following the original paper (Wang et al., 2022a) as the source codes were not publicly available when we conducted our experiments. To ensure that the training framework is the same across different approaches, we followed the official source code of Sub-LN and NormFormer, and re-implemented them on Fairseq. Other results are directly cited from corresponding papers.

Results on WMT17 En-De. Table 1 reports the results of baselines and our approach on the WMT

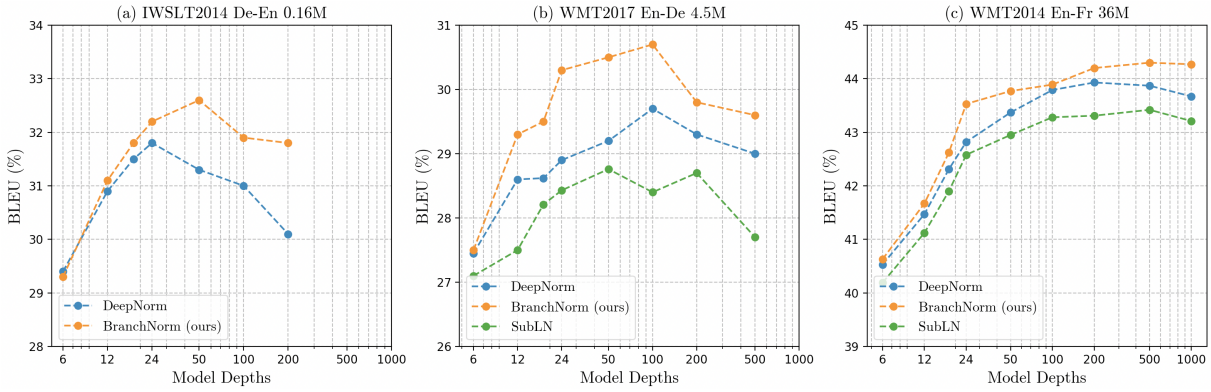

Figure 5: Performance of bilingual translation with different model depths, which are plotted on a logarithmic scale.

2017 En-De dataset. In most cases, the training of vanilla Post-LN Transformer get diverged due to its own training instability. Meanwhile, previous approaches can stabilize the training of deep Transformer to varying degrees. Results of our re-implemented DeepNorm slightly outperform those reported in the original paper, and serve as a stronger baseline to make the improvement of our approach more convincing. It is noteworthy that all approaches show different degradation of BLEU score after 200 layers. We preliminarily speculate that this phenomenon is caused by the overfitting on the small-scale WMT17 En-De after the model deepening. In summary, our BranchNorm achieves the best results consistently at different depths and mitigates the performance degradation problem mentioned above Moreover, BranchNorm outperforms previous state-of-the-art deep models by up to +1.2 BLEU given the same model depths. As shown in Figure 4, BranchNorm exhibits faster convergence and better convergence performance than DeepNorm.

Results on WMT14 En-Fr. Results of baselines and BranchNorm on the larger WMT 2014 En-Fr dataset are reported in Table 2. We observe similar findings with WMT 2014 En-De, namely, BranchNorm bring consistent improvements on models with different depths. Notably, our 500 layer model outperforms existing deep models and achieves a new SOTA performance of 44.3 BLEU.

Effects of Data Scale. We draw the detailed performance of three datasets with different scales in Figure 5. Overall, we observe that as the model deepens, performance on smaller data is compromised, while larger datasets continue to benefit from the scaling of depth. This indicates that deeper models tend to require larger data to fit, which is consistent with findings on large-scale pretraining (Hoffmann et al., 2022).

Effects of Training Steps. In Figure 4, we plot the training curves of BranchNorm and DeepNorm for the 50L-50L model on the WMT2017 En-De. The results demonstrate that BranchNorm can effectively unleash unleash the potential performance of deep models and finally yields a better converge performance. In contrast, DeepNorm suffers from the undertraining problem, and performance is harmed to a certain extent at larger training steps.

4.4 Multilingual Translation Tasks

Results of various models on the OPUS-100 and MultiUN datasets are listed in Table 3. As the depth increases from 12 to 1000, the BLEU scores are increased by +7.8 and +6.1 points respectively on the OPUS dataset and MultiUN dataset. Scaling the vanilla Pre-LN Transformer to 200 and 1000 layers proves to be ineffective, indicating that the vanilla Pre-LN Transformer is not effective enough on the deep model. BranchNorm consistently outperforms DeepNorm across all depths which is coherent with the conclusions of the bilingual translations.

5 Analysis

In this section, we first verify the robustness of BranchNorm to hyperparameter, and then analyze the parameter redundancy.

5.1 Hyperparameter Sensitivity

Effects of Different T. We conduct experiments to evaluate the effect of varying the different maximum norm step T in Equation (8) on BranchNorm.

Models	# Layers	# Params	OPUS100			MultiUN		
			X→En	En→X	Avg	X→En	En→X	Avg
Baseline (Zhang et al., 2020)	12	133M	27.5	21.4	24.5	43.8	52.3	48.1
	24	173M	29.5	22.9	26.2	46.1	53.9	50
	48	254M	31.4	24.0	27.7	–	–	–
Pre-LN(Vaswani et al., 2017)	200	863M	34.6	26.4	30.5	49.1	56.3	52.7
	1000	3.8B	34.0	28.0	31.0	50.1	56.7	53.4
DeepNorm(Wang et al., 2022a)	200	863M	33.2	**29.0**	31.1	–	–	–
	1000	3.8B	33.9	**30.2**	32.1	–	–	–
DeepNorm † (2022a)	200	863M	33.9	28.2	31.1	49.2	56.9	53.1
	1000	3.8B	34.8	29.4	32.1	50.3	57.2	53.8
BranchNorm (ours)	200	863M	**34.2**	28.5	**31.4***	**49.7**	57.2	**53.4***
	1000	3.8B	**35.0**	29.6	**32.3***	**50.8**	**57.6**	**54.2***

Table 3: Average BLEU score(%) of different models with varying depths on the OPUS-100 and MultiUN test sets. † indicates our reimplementations. The **bolded** scores correspond to the best in the same depths. '*' means BranchNorm is significantly better than DeepNorm with $p < 0.05$.

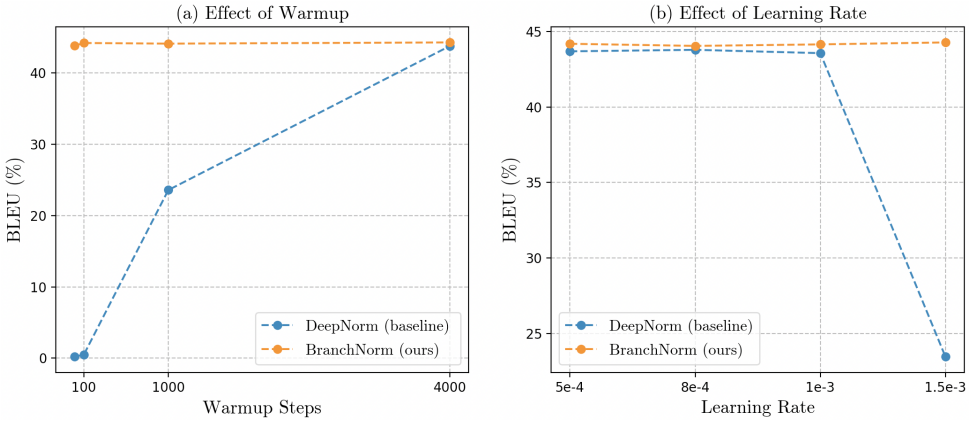

Figure 6: Effects of key hyperparameters (*i.e.,* warmup and learning rate) on training 100L-100L models on the WMT 2014 En-Fr dataset.

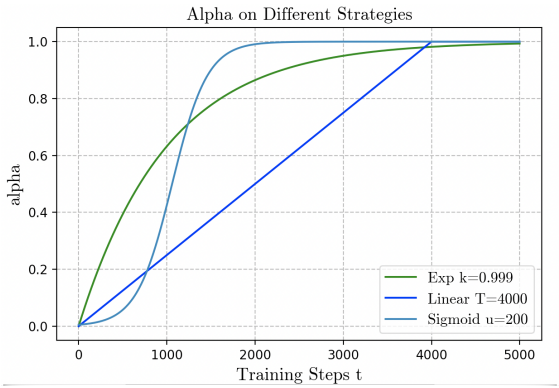

Figure 7: Different growth strategies of α in BranchNorm. Note tht α is clipped to 1.0 for all strategies.

A larger value of T corresponds to a slower degradation of BranchNorm to the vanilla Post-LN, and generally yield a more stable training process. We vary $T \in [100, 400, 4000, 20000]$ and observe that BranchNorm is insensitive to the variation of T.

Effects of Different Warmup and Learning Rate. We investigate the effect of these key hyperparameters on a 200-layer (*i.e.,* 100L-100L) Transformer on WMT14 En-Fr dataset and present the results in Figure 6. Our observations indicate that BranchNorm is able to stably train a 200-layers Transformer without the use of warmup, and exhibits better tolerance for larger learning rates when compared to DeepNorm.

Effects of Different Growing Strategies of α. We investigate the effects of various growing strategies including the default linear strategy, which is illustrated in Figure 7. For 200-layer Transformers on WMT14 En-Fr, we respectively obtain 44.20, 44.15, and 44.21 BLEU on the linear, exp, and

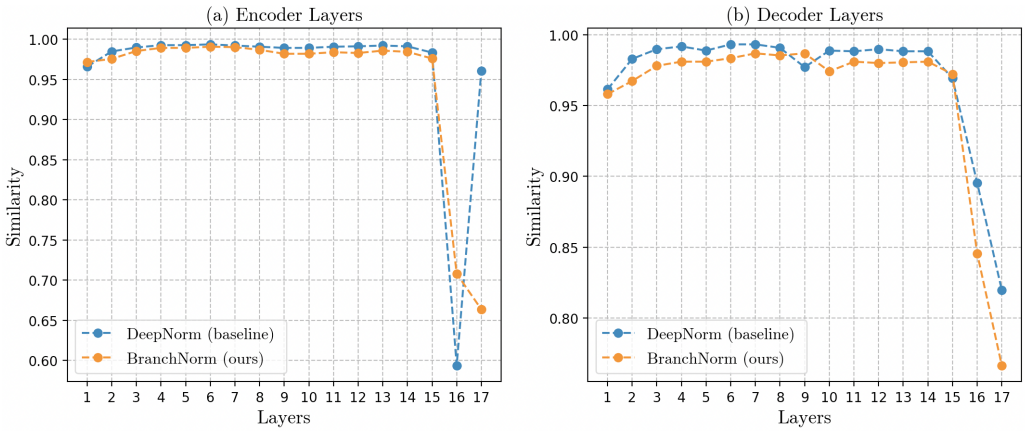

Figure 8: Representation similarity between adjacent layers. BranchNorm's values are lower than DeepNorm in most layers.

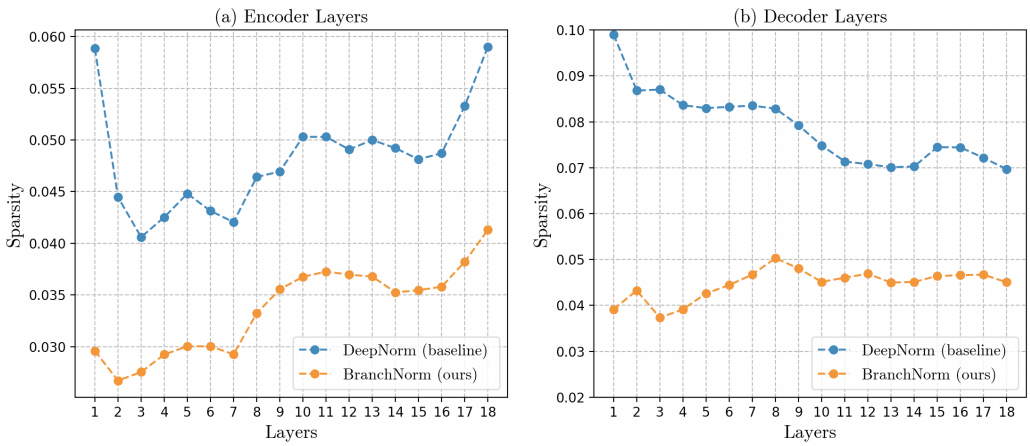

Figure 9: The sparsity of activation function of DeepNorm and BranchNorm models. The BranchNorm model is sparser than the DeepNorm one in all layers.

sigmoid strategies, indicating that our method is robust to these strategy variants, therefore, we employ the simplest linear strategy in all experiments.

5.2 Parameter Redundancy

Representation Similarity. Previous studies (Liu et al., 2020a) has posited that the Pre-LN Transformer has disproportionately large weights on its residual branch, which may inhibit its potential performance as the model deepens. Our hypothesis is that DeepNorm directly augment the weights of the residual branches in order to enhance the stability of deep model training, but may also impede the potential of the deep model. In order to verify this assumption, we employed a methodology to determine the cosine similarity of representations between adjacent layers in 200-layer (*i.e.,* 100L-100L) models that were respectively trained with DeepNorm and BranchNorm.

The representation similarity of both encoder and decoder layers is presented in Figure 8. It is observed that the similarity score of DeepNorm consistently exceeds that of BranchNorm, indicating that the augmentation of the weights of the residual branch results in the model becoming more akin to the Pre-LN Transformer. Similar findings about sparsity are consistently observed for models with different depths and data. This characteristic may subsequently contribute to the degradation of the deep model and negatively impact performance. Therefore, it is essential to revert the weights to their original values, as is done in the implementation of BranchNorm.

Sparsity of Activation Function. Li et al. (2022) studies the activation function sparsity of the Transformer and demonstrates that the sparser model comes with better generalization and robust-

ness. The sparsity is quantified by the percentage of nonzero entries after the activation function. As shown in Figure 9, we observe the sparsity of two models trained with DeepNorm and BranchNorm respectively, and find that BranchNorm had a relatively smaller sparsity. To confirm the effect of sparsity on the robustness and generalization of the model, we conducted further experiments on the MTNT (Michel and Neubig, 2018). MTNT (Machine Translation of Noisy Text) consists of noisy comments on Reddit (www.reddit.com) and professionally sourced translations and is a testbed for robust translation. We evaluate two En-Fr models that are trained with DeepNorm and BranchNorm on this noise dataset. BranchNorm has a significant improvement of 1.0 BLEU over DeepNorm, indicating that our method is able to improve robustness by increasing the sparsity of the model.

6 Conclusion

In this paper, we first explore the undertraining problem of DeepNorm and propose a more flexible canonical approach, namely BranchNorm, which theoretically stabilizes the training with smooth gradient norms at the early stage. Once the dangerous phase of training instability is passed, BranchNorm can then degenerate to a standard Post-LN, thus encouraging better convergence performance. Experiment results on several translation tasks show that BranchNorm achieves a better trade-off between training stability and converge performance.

Limitations

The training of deep Transformers generally requires large GPU resources, for example, training a 1,000-layer WMT14 En-Fr translation model requires 1000 GPU days. In addition, deeper decoders can lead to slower inference, and more model architecture design or compression techniques need to be further explored to make deep models practically deployable for applications.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *stat*, 1050:21.

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Huanru Henry Mao, Garrison W. Cottrell, and Julian J. McAuley. 2020. Rezero is all you need: Fast convergence at large depth. *CoRR*, abs/2003.04887.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT 2019*, pages 4171–4186.

Naman Goyal, Jingfei Du, Myle Ott, Giri Anantharaman, and Alexis Conneau. 2021. Larger-scale transformers for multilingual masked language modeling. *CoRR*, abs/2105.00572.

Jiatao Gu, Yong Wang, Kyunghyun Cho, and Victor OK Li. 2019. Improved zero-shot neural machine translation via ignoring spurious correlations. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1258–1268.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

Xiao Shi Huang, Felipe Pérez, Jimmy Ba, and Maksims Volkovs. 2020. Improving transformer optimization through better initialization. In *ICML 2020*, volume 119 of *Proceedings of Machine Learning Research*, pages 4475–4483.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature*, 521(7553):436–444.

Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, et al. 2022. Large models are parsimonious learners: Activation sparsity in trained transformers. *arXiv preprint arXiv:2210.06313*.

Junyang Lin, An Yang, Jinze Bai, Chang Zhou, Le Jiang, Xianyan Jia, Ang Wang, Jie Zhang, Yong Li, Wei Lin, Jingren Zhou, and Hongxia Yang. 2021. M6-10T: A sharing-delinking paradigm for efficient multi-trillion parameter pretraining. *CoRR*, abs/2110.03888.

Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. 2020a. On the variance of the adaptive learning rate and beyond. In *ICLR 2020*.

Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. 2020b. Understanding the difficulty of training transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5747–5763, Online. Association for Computational Linguistics.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692.

Paul Michel and Graham Neubig. 2018. MTNT: A testbed for machine translation of noisy text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 543–553, Brussels, Belgium. Association for Computational Linguistics.

Toan Q. Nguyen and Julian Salazar. 2019. Transformers without tears: Improving the normalization of self-attention. *CoRR*, abs/1910.05895.

Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli. 2019. fairseq: A fast, extensible toolkit for sequence modeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Demonstrations*, pages 48–53. Association for Computational Linguistics.

Sam Shleifer, Jason Weston, and Myle Ott. 2021. Normformer: Improved transformer pretraining with extra normalization. *CoRR*, abs/2110.09456.

Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS 2017*, pages 5998–6008.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. 2022a. DeepNet: Scaling Transformers to 1,000 layers. *CoRR*, abs/2203.00555.

Hongyu Wang, Shuming Ma, Shaohan Huang, Li Dong, Wenhui Wang, Zhiliang Peng, Yu Wu, Payal Bajaj, Saksham Singhal, Alon Benhaim, Barun Patra, Zhun Liu, Vishrav Chaudhary, Xia Song, and Furu Wei. 2022b. Foundation Transformers. *CoRR*, abs/2210.06423.

Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. 2019. Learning deep transformer models for machine translation. In *ACL 2019*, pages 1810–1822.

Weizhi Wang, Zhirui Zhang, Yichao Du, Boxing Chen, Jun Xie, and Weihua Luo. 2021. Rethinking zero-shot neural machine translation: From a perspective of latent variables. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4321–4327, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Biao Zhang, Ivan Titov, and Rico Sennrich. 2019a. Improving deep transformer with depth-scaled initialization and merged attention. In *EMNLP-IJCNLP 2019*, pages 898–909.

Biao Zhang, Philip Williams, Ivan Titov, and Rico Sennrich. 2020. Improving massively multilingual neural machine translation and zero-shot translation. In *ACL 2020*, pages 1628–1639. Association for Computational Linguistics.

Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma. 2019b. Fixup initialization: Residual learning without normalization. In *ICLR 2019*.

Chen Zhu, Renkun Ni, Zheng Xu, Kezhi Kong, W Ronny Huang, and Tom Goldstein. 2021. Gradinit: Learning to initialize neural networks for stable and efficient training. *Advances in Neural Information Processing Systems*, 34:16410–16422.

A Theoretical Proofs

A.1 Gradients of Post-LN

Given a Transformer with L sub-layers and the training loss $\mathcal{E}$, the gradient for the l-th sub-layer is calculated by the chain rule:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \frac{\partial \mathcal{E}}{\partial x_L} \frac{\partial x_L}{\partial x_l} \tag{10}$$

Recursively decomposing $\frac{\partial x_L}{\partial x_l}$ in the above equation, we have:

$$\frac{\partial x_L}{\partial x_l} = \frac{\partial x_L}{\partial x_{L-1}} \frac{\partial x_{L-1}}{\partial x_{L-2}} \cdots \frac{\partial x_{l+1}}{\partial x_l} \tag{11}$$

Given the Post-LN calculate the x_{l+1} as :

$$x_{l+1} = LN(x_l + \mathcal{F}(x_l; \theta_l)) \tag{12}$$

If we name the output of residual connection as $y_l = x_l + \mathcal{F}(x_l; \theta_l)$, we can calculate the partial derivatives of two adjacent layers as:

$$\begin{aligned} \frac{\partial x_{l+1}}{\partial x_l} &= \frac{\partial x_{l+1}}{\partial y_l} \frac{\partial y_l}{\partial x_l} \\ &= \frac{\partial \mathrm{LN}(y_l)}{\partial y_l} \left(1 + \frac{\partial \mathcal{F}(x_l; \theta_l)}{\partial x_l}\right) \end{aligned} \tag{13}$$

We put Equation (13) and Equation (11) into Equation (10) and get:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(y_k\right)}{\partial y_k}}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}\left(x_k;\theta_k\right)}{\partial x_k}\right)}_{residual} \tag{14}$$

the above gradient consists of three terms and the last two items are multiplications with respect to the number of model layers L. Once L get larger, the gradient of Post-LN will face the risk of vanishing or exploding.

A.2 Gradients of DeepNorm

DeepNorm rescales the residual branch with a scalar multiplier $\alpha > 1$, and calculates the sub-layer as follows:

$$x_{l+1} = LN(\alpha x_l + \mathcal{F}\left(x_l;\theta_l\right)) \tag{15}$$

Follow the above process in A.1, we have the gradient of DeepNorm:

$$\frac{\partial \mathcal{E}}{\partial x_l} = \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \left(\frac{\partial \mathrm{LN}\left(\alpha x_k + \mathcal{F}\left(x_k;\theta_l\right)\right)}{\partial\left(\alpha x_k + \mathcal{F}\left(x_k;\theta_l\right)\right)}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \left(\alpha + \frac{\partial \mathcal{F}\left(x_k;\theta_k\right)}{\partial x_k}\right)}_{residual} \tag{16}$$

Given that DeepNorm assigns a relative larger value for α to make it to amplify the output percentage of residual connections. Here, we introduce an assumption to simplify the derivation: If α gets large enough, we can approximate the above equation as follows:

$$\frac{\partial \mathcal{E}}{\partial x_l} \approx \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \left(\frac{\partial \mathrm{LN}\left(\alpha x_k\right)}{\partial\left(\alpha x_k\right)}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \alpha}_{residual} \tag{17}$$

We let $z_k = \alpha x_k$ and use the chain rule, then get:

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial x_l} &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \left(\frac{\partial \mathrm{LN}\left(z_k\right)}{\partial x_k} \times \frac{\partial x_k}{\partial z_k}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \alpha}_{residual} \\ &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \left(\frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k} \times \frac{1}{\alpha}\right)}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \alpha}_{residual} \\ &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k}}_{LN}\end{aligned} \tag{18}$$

When compared with the gradient of Post-LN in Equation (14), DeepNorm can approximately eliminate the final multiplication item, and thus mitigate the risk of gradient vanishing or exploding to a certain degree.

A.3 Gradients of BranchNorm

BranchNorm directly rescale the non-residual branch in Transformer and conduct calculations for the l-th sub-layer as:

$$x_{l+1} = LN(x_l + \alpha\mathcal{F}\left(x_l;\theta_l\right)) \tag{19}$$

Similar to the previous analysis process, we can calculate the gradients of BranchNorm as:

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial x_l} &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k + \alpha\mathcal{F}\left(x_k;\theta_l\right)\right)}{\partial\left(x_k + \alpha\mathcal{F}\left(x_k;\theta_l\right)\right)}}_{LN} \times \underbrace{\prod_{k=l}^{L-1} \left(1 + \alpha\frac{\partial \mathcal{F}\left(x_k;\theta_l\right)}{\partial x_k}\right)}_{residual} \\ &= \underbrace{\frac{\partial \mathcal{E}}{\partial x_L}}_{irreducible} \times \underbrace{\prod_{k=l}^{L-1} \frac{\partial \mathrm{LN}\left(x_k\right)}{\partial x_k}}_{LN} \quad (\alpha = 0)\end{aligned} \tag{20}$$

BranchNorm can stabilize the gradient norm into while DeepNorm require a relatively strong assumption in Equation (6). Experimentally, in Figure 3, we observe corresponding smoother gradients of BranchNorm at the very beginning of training.

B Hyperparameter

Hyperparameters	**Small Scale**	**Medium Scale**	**Large Scale**
Learning rate		5e-4	
Learning rate scheduler		inverse sqrt	
Warm-up updates		4000	
Warm-up init learning rate		1e-7	
Max tokens		128×4096	
Adam ϵ		1e-8	
Adam β		(0.9, 0.98)	
Label smoothing		0.1	
Training updates		100K	
Gradient clipping		0.0	
Dropout	0.4	0.2	0.1
Weight decay		0.0001	
Hidden size		512	
FFN inner hidden size		2048	
Attention heads		8	

Table 4: Hyperparameters for the Transformer$_{base}$ experiments on different data sizes. 'Small Scale': IWSLT 2014 De-En and WMT17 En-De. 'Medium Scale': WMT14 En-Fr. 'Large Scale': OPUS-100 and MultiUN datasets.